Introduction to Theoretical Computer Science

CS2490 - Session 28: From PDA to CFG

Pierre Cagne

Fall 2026

October 23, 2026

Last time...

Theorem

A language is context-free if and only if it is recognized by a PDA.

Last time...

Theorem

A language is context-free if and only if it is recognized by a PDA.

We have seen an algorithm for $\Longrightarrow$:

Last time...

Theorem

A language is context-free if and only if it is recognized by a PDA.

We have seen an algorithm for $\Longrightarrow$: put the CFG $G = (\Sigma, V, S, P)$ in Generalized Chomsky Normal Form,

Last time...

Theorem

A language is context-free if and only if it is recognized by a PDA.

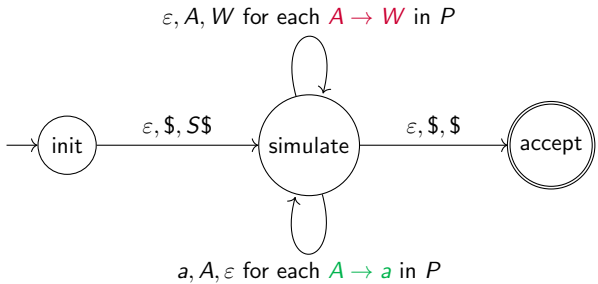
We have seen an algorithm for $\Longrightarrow$: put the CFG $G = (\Sigma, V, S, P)$ in Generalized Chomsky Normal Form, then use the stack to simulate its leftmost derivations:

Last time...

Theorem

A language is context-free if and only if it is recognized by a PDA.

We have seen an algorithm for $\Longrightarrow$: put the CFG $G = (\Sigma, V, S, P)$ in Generalized Chomsky Normal Form, then use the stack to simulate its leftmost derivations:

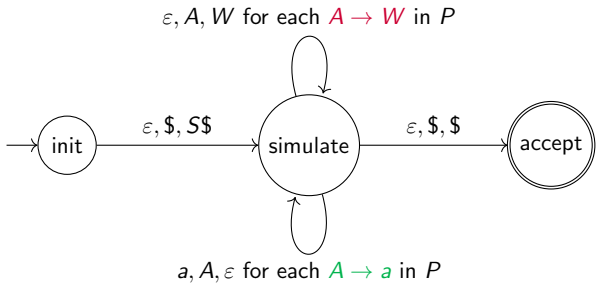


Last time...

Theorem

A language is context-free if and only if it is recognized by a PDA.

We have seen an algorithm for $\Longrightarrow$: put the CFG $G = (\Sigma, V, S, P)$ in Generalized Chomsky Normal Form, then use the stack to simulate its leftmost derivations:



Today: the converse $\Longleftarrow$, i.e., turning a PDA into a CFG.

Transforming a PDA into a CFG: Ginsburg triples

Let $\mathcal{A} = (\Sigma, Q, \Gamma, \Delta, q_0, z, F)$ be a PDA.

Transforming a PDA into a CFG: Ginsburg triples

Let $\mathcal{A} = (\Sigma, Q, \Gamma, \Delta, q_0, z, F)$ be a PDA. Let's construct a CFG $G = (\Sigma, V, S, P)$ that generates $\mathrm{L}(\mathcal{A})$.

Transforming a PDA into a CFG: Ginsburg triples

Let $\mathcal{A} = (\Sigma, Q, \Gamma, \Delta, q_0, z, F)$ be a PDA. Let's construct a CFG $G = (\Sigma, V, S, P)$ that generates $\mathrm{L}(\mathcal{A})$.

We make a first "restriction" on $\mathcal{A}$: it has a single accepting state q_f, distinct from q_0, no transition goes out of q_f, and any transition entering q_f is of the form a, z, ε with $a \in \Sigma \cup \{\varepsilon\}$.

Transforming a PDA into a CFG: Ginsburg triples

Let $\mathcal{A} = (\Sigma, Q, \Gamma, \Delta, q_0, z, F)$ be a PDA. Let's construct a CFG $G = (\Sigma, V, S, P)$ that generates $\mathrm{L}(\mathcal{A})$.

We make a first "restriction" on $\mathcal{A}$: it has a single accepting state q_f, distinct from q_0, no transition goes out of q_f, and any transition entering q_f is of the form a, z, ε with $a \in \Sigma \cup \{\varepsilon\}$. (That is, the stack must see the bottom to enter the final state, and we leave the stack completely empty).

Transforming a PDA into a CFG: Ginsburg triples

Let $\mathcal{A} = (\Sigma, Q, \Gamma, \Delta, q_0, z, F)$ be a PDA. Let's construct a CFG $G = (\Sigma, V, S, P)$ that generates $\mathrm{L}(\mathcal{A})$.

We make a first "restriction" on $\mathcal{A}$: it has a single accepting state q_f, distinct from q_0, no transition goes out of q_f, and any transition entering q_f is of the form a, z, ε with $a \in \Sigma \cup \{\varepsilon\}$. (That is, the stack must see the bottom to enter the final state, and we leave the stack completely empty).

Idea: each triple (p, A, q), with $p, q \in Q$ and $A \in \Gamma$, is a variable of G; every derivation from (p, A, q) corresponds to a computation of $\mathcal{A}$ that starts in state p with A on top of the stack and ends in state q with A popped from the stack.

Transforming a PDA into a CFG: Ginsburg triples

Let $\mathcal{A} = (\Sigma, Q, \Gamma, \Delta, q_0, z, F)$ be a PDA. Let's construct a CFG $G = (\Sigma, V, S, P)$ that generates $\mathrm{L}(\mathcal{A})$.

We make a first "restriction" on $\mathcal{A}$: it has a single accepting state q_f, distinct from q_0, no transition goes out of q_f, and any transition entering q_f is of the form a, z, ε with $a \in \Sigma \cup \{\varepsilon\}$. (That is, the stack must see the bottom to enter the final state, and we leave the stack completely empty).

Idea: each triple (p, A, q), with $p, q \in Q$ and $A \in \Gamma$, is a variable of G; every derivation from (p, A, q) corresponds to a computation of $\mathcal{A}$ that starts in state p with A on top of the stack and ends in state q with A popped from the stack.

Formally, we aim for: $(p, A, q) \Rightarrow^* w \iff \forall \sigma \in \Gamma^*, (p, w, A\sigma) \vdash^* (q, \varepsilon, \sigma)$.

Transforming a PDA into a CFG: Ginsburg triples

Let $\mathcal{A} = (\Sigma, Q, \Gamma, \Delta, q_0, z, F)$ be a PDA. Let's construct a CFG $G = (\Sigma, V, S, P)$ that generates $\mathsf{L}(\mathcal{A})$.

We make a first "restriction" on $\mathcal{A}$: it has a single accepting state q_f, distinct from q_0, no transition goes out of q_f, and any transition entering q_f is of the form a, z, ε with $a \in \Sigma \cup \{\varepsilon\}$. (That is, the stack must see the bottom to enter the final state, and we leave the stack completely empty).

Idea: each triple (p, A, q), with $p, q \in Q$ and $A \in \Gamma$, is a variable of G; every derivation from (p, A, q) corresponds to a computation of $\mathcal{A}$ that starts in state p with A on top of the stack and ends in state q with A popped from the stack.

Formally, we aim for: $(p, A, q) \Rightarrow^* w \iff \forall \sigma \in \Gamma^*, (p, w, A\sigma) \vdash^* (q, \varepsilon, \sigma)$.

Thus, choosing (q_0, z, q_f) as a starting variable, G generates $\mathsf{L}(\mathcal{A})$. Indeed:

$$(q_0, z, q_f) \Rightarrow^* w \iff \forall \sigma, (q_0, w, z\sigma) \vdash^* (q_f, \varepsilon, \sigma) \iff (q_0, w, z) \vdash^* (q_f, \varepsilon, \varepsilon) \iff w \in \mathsf{L}(\mathcal{A}).$$

Transforming a PDA into a CFG: Ginsburg triples (cont'd)

Now, we need to define the production rules of G.

Transforming a PDA into a CFG: Ginsburg triples (cont'd)

Now, we need to define the production rules of G.

We make a second "restriction" on $\mathcal{A}$: every transition (except those entering the final state) pushes either nothing or exactly two symbols onto the stack.

Transforming a PDA into a CFG: Ginsburg triples (cont'd)

Now, we need to define the production rules of G.

We make a second "restriction" on $\mathcal{A}$: every transition (except those entering the final state) pushes either nothing or exactly two symbols onto the stack. (Each transition increments or decrements the stack height by exactly one.)

Transforming a PDA into a CFG: Ginsburg triples (cont'd)

Now, we need to define the production rules of G.

We make a second "restriction" on $\mathcal{A}$: every transition (except those entering the final state) pushes either nothing or exactly two symbols onto the stack. (Each transition increments or decrements the stack height by exactly one.)

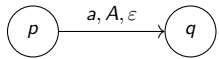
Idea:

Transforming a PDA into a CFG: Ginsburg triples (cont'd)

Now, we need to define the production rules of G.

We make a second "restriction" on $\mathcal{A}$: every transition (except those entering the final state) pushes either nothing or exactly two symbols onto the stack. (Each transition increments or decrements the stack height by exactly one.)

Idea:

- For each transition of the form: 

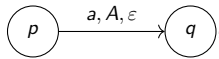
, add the production rule $(p, A, q) \to a$

Transforming a PDA into a CFG: Ginsburg triples (cont'd)

Now, we need to define the production rules of G.

We make a second "restriction" on $\mathcal{A}$: every transition (except those entering the final state) pushes either nothing or exactly two symbols onto the stack. (Each transition increments or decrements the stack height by exactly one.)

Idea:

- For each transition of the form: 

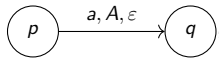
, add the production rule $(p, A, q) \to a$ to mimic the computation step $(p, a, A\sigma) \vdash (q, \varepsilon, \sigma)$ for some $\sigma \in \Gamma^*$.

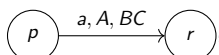
Transforming a PDA into a CFG: Ginsburg triples (cont'd)

Now, we need to define the production rules of G.

We make a second "restriction" on $\mathcal{A}$: every transition (except those entering the final state) pushes either nothing or exactly two symbols onto the stack. (Each transition increments or decrements the stack height by exactly one.)

Idea:

- For each transition of the form: 

, add the production rule $(p, A, q) \to a$ to mimic the computation step $(p, a, A\sigma) \vdash (q, \varepsilon, \sigma)$ for some $\sigma \in \Gamma^*$.
- For each transition of the form: 

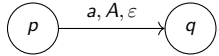
, add the production rules $(p, A, q) \to a(r, B, t)(t, C, q)$ for all $q, t \in Q$

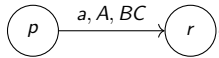
Transforming a PDA into a CFG: Ginsburg triples (cont'd)

Now, we need to define the production rules of G.

We make a second "restriction" on $\mathcal{A}$: every transition (except those entering the final state) pushes either nothing or exactly two symbols onto the stack. (Each transition increments or decrements the stack height by exactly one.)

Idea:

- For each transition of the form: 

, add the production rule $(p, A, q) \to a$ to mimic the computation step $(p, a, A\sigma) \vdash (q, \varepsilon, \sigma)$ for some $\sigma \in \Gamma^*$.
- For each transition of the form: 

, add the production rules $(p, A, q) \to a(r, B, t)(t, C, q)$ for all $q, t \in Q$ to mimic the first step of a computation of the form:

$$(p, auv, A\sigma) \vdash (r, uv, BC\sigma) \vdash^* (t, v, C\sigma) \vdash^* (q, \varepsilon, \sigma) \text{ for some } t \in Q, u, v \in \Sigma^*, \sigma \in \Gamma^*.$$

Restrictions? No problem!

Any PDA can be transformed into one that meets the two "restrictions".

Restrictions? No problem!

Any PDA can be transformed into one that meets the two "restrictions".

If there is already only one accepting state, distinct from q_0, and all transitions entering it are of the form a, z, z, then replace each such transition by a, z, ε.

Restrictions? No problem!

Any PDA can be transformed into one that meets the two "restrictions".

If there is already only one accepting state, distinct from q_0, and all transitions entering it are of the form a, z, z, then replace each such transition by a, z, ε.

Otherwise, to ensure the first "restriction":

Restrictions? No problem!

Any PDA can be transformed into one that meets the two "restrictions".

If there is already only one accepting state, distinct from q_0, and all transitions entering it are of the form a, z, z, then replace each such transition by a, z, ε.

Otherwise, to ensure the first "restriction":

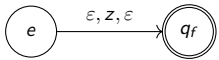
- add two new states e and q_f,

Restrictions? No problem!

Any PDA can be transformed into one that meets the two "restrictions".

If there is already only one accepting state, distinct from q_0, and all transitions entering it are of the form a, z, z, then replace each such transition by a, z, ε.

Otherwise, to ensure the first "restriction":

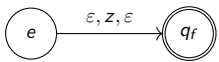
- add two new states e and q_f,
- add the transition 

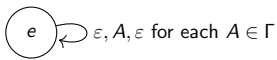
Restrictions? No problem!

Any PDA can be transformed into one that meets the two "restrictions".

If there is already only one accepting state, distinct from q_0, and all transitions entering it are of the form a, z, z, then replace each such transition by a, z, ε.

Otherwise, to ensure the first "restriction":

- add two new states e and q_f,
- add the transition 

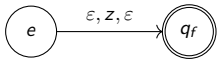
- add the transitions 

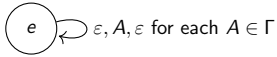
Restrictions? No problem!

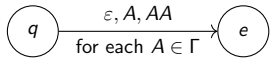
Any PDA can be transformed into one that meets the two "restrictions".

If there is already only one accepting state, distinct from q_0, and all transitions entering it are of the form a, z, z, then replace each such transition by a, z, ε.

Otherwise, to ensure the first "restriction":

- add two new states e and q_f,
- add the transition 

- add the transitions 

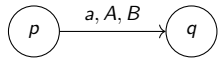
- every former final state $q \in F$ now transitions to e via 

Restrictions? No problem! (cont'd)

To ensure the second "restriction":

Restrictions? No problem! (cont'd)

To ensure the second "restriction":

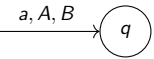
- replace each transition of the form 

by

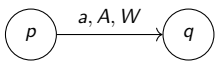


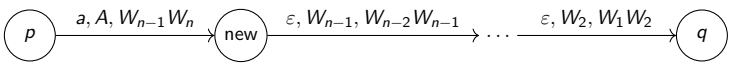
Restrictions? No problem! (cont'd)

To ensure the second "restriction":

- replace each transition of the form 

by


- replace each transition of the form 

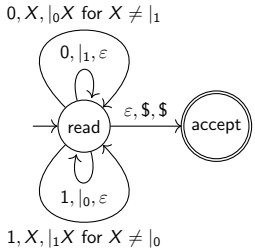
where $|W| > 2$ by a sequence of transitions that push two symbols at a time:


Example

Let's transform the PDA recognizing the binary words with an equal number of 0s and 1s into a CFG using Ginsburg triples!

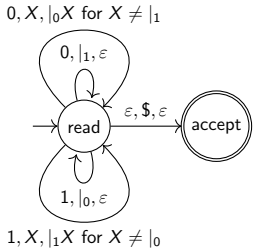
Example

Let's transform the PDA recognizing the binary words with an equal number of 0s and 1s into a CFG using Ginsburg triples!



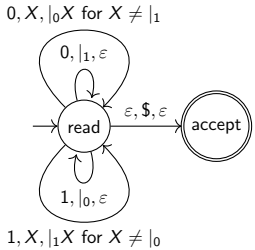
Example

Let's transform the PDA recognizing the binary words with an equal number of 0s and 1s into a CFG using Ginsburg triples!



Example

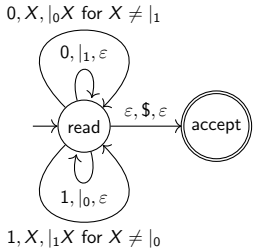
Let's transform the PDA recognizing the binary words with an equal number of 0s and 1s into a CFG using Ginsburg triples!



The grammar has variables (p, A, q) for every $p, q \in \{\text{read}, \text{accept}\}$ and $A \in \{\$, |_0, |_1\}$.

Example

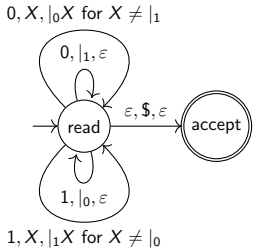
Let's transform the PDA recognizing the binary words with an equal number of 0s and 1s into a CFG using Ginsburg triples!



The grammar has variables (p, A, q) for every $p, q \in \{\text{read}, \text{accept}\}$ and $A \in \{\$, |_0, |_1\}$. (So there are $2 \times 2 \times 3 = 12$ variables.)

Example

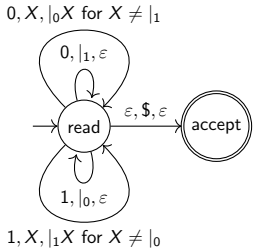
Let's transform the PDA recognizing the binary words with an equal number of 0s and 1s into a CFG using Ginsburg triples!



The grammar has variables (p, A, q) for every $p, q \in \{\text{read}, \text{accept}\}$ and $A \in \{\$, |_0, |_1\}$. (So there are $2 \times 2 \times 3 = 12$ variables.) The starting variable is $(\text{read}, \$, \text{accept})$.

Example

Let's transform the PDA recognizing the binary words with an equal number of 0s and 1s into a CFG using Ginsburg triples!



The grammar has variables (p, A, q) for every $p, q \in \{\text{read}, \text{accept}\}$ and $A \in \{\$, |_0, |_1\}$. (So there are $2 \times 2 \times 3 = 12$ variables.) The starting variable is $(\text{read}, \$, \text{accept})$.

Let's write the *easy* production rules first:

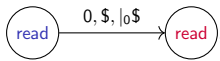
$$(\text{read}, \$, \text{accept}) \to \varepsilon, \quad (\text{read}, |_1, \text{read}) \to 0, \quad \text{and} \quad (\text{read}, |_0, \text{read}) \to 1.$$

Example (cont'd)

Let's now tackle the more *involved* production rules:

Example (cont'd)

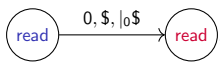
Let's now tackle the more *involved* production rules:

For the transition 

, we add 2×2 production rules:

Example (cont'd)

Let's now tackle the more *involved* production rules:

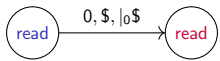
For the transition 

, we add 2×2 production rules:

$$(\text{read}, \$, \text{any state}) \rightarrow 0(\text{read}, |_0, \text{any state})(\text{any state}, \$, \text{any state})$$

Example (cont'd)

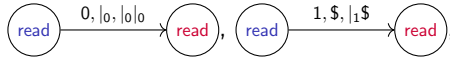
Let's now tackle the more *involved* production rules:

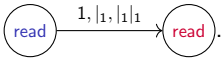
For the transition 

, we add 2×2 production rules:

$$(\text{read}, \$, \text{read}) \to 0(\text{read}, |_0, \text{read})(\text{read}, \$, \text{read}),$$
$$(\text{read}, \$, \text{accept}) \to 0(\text{read}, |_0, \text{read})(\text{read}, \$, \text{accept}),$$
$$(\text{read}, \$, \text{read}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, \$, \text{read}),$$
$$(\text{read}, \$, \text{accept}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, \$, \text{accept}).$$

Example (cont'd)

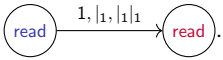
We do this for all the remaining transitions: 

, and 

.

Example (cont'd)

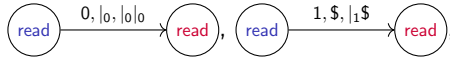
We do this for all the remaining transitions: 

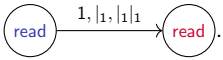
, and 

.

This gives:

Example (cont'd)

We do this for all the remaining transitions: 

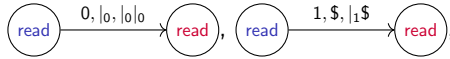
, and 

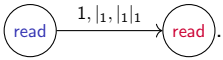
.

This gives:

$(\text{read}, |_0, \text{read}) \to 0(\text{read}, |_0, \text{read})(\text{read}, |_0, \text{read}),$
$(\text{read}, |_0, \text{accept}) \to 0(\text{read}, |_0, \text{read})(\text{read}, |_0, \text{accept}),$
$(\text{read}, |_0, \text{read}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, |_0, \text{read}),$
$(\text{read}, |_0, \text{accept}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, |_0, \text{accept}).$

Example (cont'd)

We do this for all the remaining transitions: 

, and 

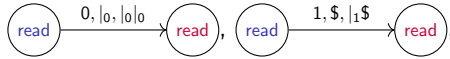
.

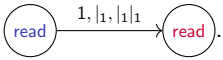
This gives:

$(\text{read}, |_0, \text{read}) \to 0(\text{read}, |_0, \text{read})(\text{read}, |_0, \text{read}),$
$(\text{read}, |_0, \text{accept}) \to 0(\text{read}, |_0, \text{read})(\text{read}, |_0, \text{accept}),$
$(\text{read}, |_0, \text{read}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, |_0, \text{read}),$
$(\text{read}, |_0, \text{accept}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, |_0, \text{accept}).$

$(\text{read}, \$, \text{read}) \to 1(\text{read}, |_1, \text{read})(\text{read}, \$, \text{read}),$
$(\text{read}, \$, \text{accept}) \to 1(\text{read}, |_1, \text{read})(\text{read}, \$, \text{accept}),$
$(\text{read}, \$, \text{read}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, \$, \text{read}),$
$(\text{read}, \$, \text{accept}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, \$, \text{accept}).$

Example (cont'd)

We do this for all the remaining transitions: 

, and 

.

This gives:

$(\text{read}, |_0, \text{read}) \to 0(\text{read}, |_0, \text{read})(\text{read}, |_0, \text{read})$,
$(\text{read}, |_0, \text{accept}) \to 0(\text{read}, |_0, \text{read})(\text{read}, |_0, \text{accept})$,
$(\text{read}, |_0, \text{read}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, |_0, \text{read})$,
$(\text{read}, |_0, \text{accept}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, |_0, \text{accept})$.

$(\text{read}, |_1, \text{read}) \to 1(\text{read}, |_1, \text{read})(\text{read}, |_1, \text{read})$,
$(\text{read}, |_1, \text{accept}) \to 1(\text{read}, |_1, \text{read})(\text{read}, |_1, \text{accept})$,
$(\text{read}, |_1, \text{read}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, |_1, \text{read})$,
$(\text{read}, |_1, \text{accept}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, |_1, \text{accept})$.

$(\text{read}, \$, \text{read}) \to 1(\text{read}, |_1, \text{read})(\text{read}, \$, \text{read})$,
$(\text{read}, \$, \text{accept}) \to 1(\text{read}, |_1, \text{read})(\text{read}, \$, \text{accept})$,
$(\text{read}, \$, \text{read}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, \$, \text{read})$,
$(\text{read}, \$, \text{accept}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, \$, \text{accept})$.

Example (cont'd)

In the end, we have the following production rules. Let's trim them!

$(\text{read}, \$, \text{accept}) \to \varepsilon,$
$(\text{read}, |_1, \text{read}) \to 0,$
$(\text{read}, |_0, \text{read}) \to 1,$
$(\text{read}, \$, \text{read}) \to 0(\text{read}, |_0, \text{read})(\text{read}, \$, \text{read}),$
$(\text{read}, \$, \text{accept}) \to 0(\text{read}, |_0, \text{read})(\text{read}, \$, \text{accept}),$
$(\text{read}, \$, \text{read}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, \$, \text{read}),$
$(\text{read}, \$, \text{accept}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, \$, \text{accept}),$
$(\text{read}, \$, \text{read}) \to 1(\text{read}, |_1, \text{read})(\text{read}, \$, \text{read}),$
$(\text{read}, \$, \text{accept}) \to 1(\text{read}, |_1, \text{read})(\text{read}, \$, \text{accept}),$
$(\text{read}, \$, \text{read}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, \$, \text{read}),$
$(\text{read}, \$, \text{accept}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, \$, \text{accept}).$

$(\text{read}, |_0, \text{read}) \to 0(\text{read}, |_0, \text{read})(\text{read}, |_0, \text{read}),$
$(\text{read}, |_0, \text{accept}) \to 0(\text{read}, |_0, \text{read})(\text{read}, |_0, \text{accept}),$
$(\text{read}, |_0, \text{read}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, |_0, \text{read}),$
$(\text{read}, |_0, \text{accept}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, |_0, \text{accept}),$
$(\text{read}, |_1, \text{read}) \to 1(\text{read}, |_1, \text{read})(\text{read}, |_1, \text{read}),$
$(\text{read}, |_1, \text{accept}) \to 1(\text{read}, |_1, \text{read})(\text{read}, |_1, \text{accept}),$
$(\text{read}, |_1, \text{read}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, |_1, \text{read}),$
$(\text{read}, |_1, \text{accept}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, |_1, \text{accept}).$

Example (cont'd)

In the end, we have the following production rules. Let's trim them!

$(\text{read}, \$, \text{accept}) \to \varepsilon$,
$(\text{read}, |_1, \text{read}) \to 0$,
$(\text{read}, |_0, \text{read}) \to 1$,
$(\text{read}, \$, \text{read}) \to 0(\text{read}, |_0, \text{read})(\text{read}, \$, \text{read})$,
$(\text{read}, \$, \text{accept}) \to 0(\text{read}, |_0, \text{read})(\text{read}, \$, \text{accept})$,
$(\text{read}, \$, \text{read}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, \$, \text{read})$,
$(\text{read}, \$, \text{accept}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, \$, \text{accept})$,
$(\text{read}, \$, \text{read}) \to 1(\text{read}, |_1, \text{read})(\text{read}, \$, \text{read})$,
$(\text{read}, \$, \text{accept}) \to 1(\text{read}, |_1, \text{read})(\text{read}, \$, \text{accept})$,
$(\text{read}, \$, \text{read}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, \$, \text{read})$,
$(\text{read}, \$, \text{accept}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, \$, \text{accept})$.

$(\text{read}, |_0, \text{read}) \to 0(\text{read}, |_0, \text{read})(\text{read}, |_0, \text{read})$,
$(\text{read}, |_0, \text{accept}) \to 0(\text{read}, |_0, \text{read})(\text{read}, |_0, \text{accept})$,
$(\text{read}, |_0, \text{read}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, |_0, \text{read})$,
$(\text{read}, |_0, \text{accept}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, |_0, \text{accept})$,
$(\text{read}, |_1, \text{read}) \to 1(\text{read}, |_1, \text{read})(\text{read}, |_1, \text{read})$,
$(\text{read}, |_1, \text{accept}) \to 1(\text{read}, |_1, \text{read})(\text{read}, |_1, \text{accept})$,
$(\text{read}, |_1, \text{read}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, |_1, \text{read})$,
$(\text{read}, |_1, \text{accept}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, |_1, \text{accept})$.

Example (cont'd)

In the end, we have the following production rules. Let's trim them!

$(\text{read}, \$, \text{accept}) \to \varepsilon$,
$(\text{read}, |_1, \text{read}) \to 0$,
$(\text{read}, |_0, \text{read}) \to 1$,
$(\text{read}, \$, \text{read}) \to 0(\text{read}, |_0, \text{read})(\text{read}, \$, \text{read})$,
$(\text{read}, \$, \text{accept}) \to 0(\text{read}, |_0, \text{read})(\text{read}, \$, \text{accept})$,
$(\text{read}, \$, \text{read}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, \$, \text{read})$,
$(\text{read}, \$, \text{accept}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, \$, \text{accept})$,
$(\text{read}, \$, \text{read}) \to 1(\text{read}, |_1, \text{read})(\text{read}, \$, \text{read})$,
$(\text{read}, \$, \text{accept}) \to 1(\text{read}, |_1, \text{read})(\text{read}, \$, \text{accept})$,
$(\text{read}, \$, \text{read}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, \$, \text{read})$,
$(\text{read}, \$, \text{accept}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, \$, \text{accept})$.

$(\text{read}, |_0, \text{read}) \to 0(\text{read}, |_0, \text{read})(\text{read}, |_0, \text{read})$,
$(\text{read}, |_0, \text{accept}) \to 0(\text{read}, |_0, \text{read})(\text{read}, |_0, \text{accept})$,
$(\text{read}, |_0, \text{read}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, |_0, \text{read})$,
$(\text{read}, |_0, \text{accept}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, |_0, \text{accept})$,
$(\text{read}, |_1, \text{read}) \to 1(\text{read}, |_1, \text{read})(\text{read}, |_1, \text{read})$,
$(\text{read}, |_1, \text{accept}) \to 1(\text{read}, |_1, \text{read})(\text{read}, |_1, \text{accept})$,
$(\text{read}, |_1, \text{read}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, |_1, \text{read})$,
$(\text{read}, |_1, \text{accept}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, |_1, \text{accept})$.

Example (cont'd)

In the end, we have the following production rules. Let's trim them!

$(\text{read}, \$, \text{accept}) \to \varepsilon$,
$(\text{read}, |_1, \text{read}) \to 0$,
$(\text{read}, |_0, \text{read}) \to 1$,
$(\text{read}, \$, \text{read}) \to 0(\text{read}, |_0, \text{read})(\text{read}, \$, \text{read})$,
$(\text{read}, \$, \text{accept}) \to 0(\text{read}, |_0, \text{read})(\text{read}, \$, \text{accept})$,
$(\text{read}, \$, \text{read}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, \$, \text{read})$,
$(\text{read}, \$, \text{accept}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, \$, \text{accept})$,
$(\text{read}, \$, \text{read}) \to 1(\text{read}, |_1, \text{read})(\text{read}, \$, \text{read})$,
$(\text{read}, \$, \text{accept}) \to 1(\text{read}, |_1, \text{read})(\text{read}, \$, \text{accept})$,
$(\text{read}, \$, \text{read}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, \$, \text{read})$,
$(\text{read}, \$, \text{accept}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, \$, \text{accept})$.

$(\text{read}, |_0, \text{read}) \to 0(\text{read}, |_0, \text{read})(\text{read}, |_0, \text{read})$,
$(\text{read}, |_0, \text{accept}) \to 0(\text{read}, |_0, \text{read})(\text{read}, |_0, \text{accept})$,
$(\text{read}, |_0, \text{read}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, |_0, \text{read})$,
$(\text{read}, |_0, \text{accept}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, |_0, \text{accept})$,
$(\text{read}, |_1, \text{read}) \to 1(\text{read}, |_1, \text{read})(\text{read}, |_1, \text{read})$,
$(\text{read}, |_1, \text{accept}) \to 1(\text{read}, |_1, \text{read})(\text{read}, |_1, \text{accept})$,
$(\text{read}, |_1, \text{read}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, |_1, \text{read})$,
$(\text{read}, |_1, \text{accept}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, |_1, \text{accept})$.

Example (cont'd)

In the end, we have the following production rules. Let's trim them!

$(\text{read}, \$, \text{accept}) \to \varepsilon$,
$(\text{read}, |_1, \text{read}) \to 0$,
$(\text{read}, |_0, \text{read}) \to 1$,
$(\text{read}, \$, \text{read}) \to 0(\text{read}, |_0, \text{read})(\text{read}, \$, \text{read})$,
$(\text{read}, \$, \text{accept}) \to 0(\text{read}, |_0, \text{read})(\text{read}, \$, \text{accept})$,
$(\text{read}, \$, \text{read}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, \$, \text{read})$,
$(\text{read}, \$, \text{accept}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, \$, \text{accept})$,
$(\text{read}, \$, \text{read}) \to 1(\text{read}, |_1, \text{read})(\text{read}, \$, \text{read})$,
$(\text{read}, \$, \text{accept}) \to 1(\text{read}, |_1, \text{read})(\text{read}, \$, \text{accept})$,
$(\text{read}, \$, \text{read}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, \$, \text{read})$,
$(\text{read}, \$, \text{accept}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, \$, \text{accept})$.

$(\text{read}, |_0, \text{read}) \to 0(\text{read}, |_0, \text{read})(\text{read}, |_0, \text{read})$,
$(\text{read}, |_0, \text{accept}) \to 0(\text{read}, |_0, \text{read})(\text{read}, |_0, \text{accept})$,
$(\text{read}, |_0, \text{read}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, |_0, \text{read})$,
$(\text{read}, |_0, \text{accept}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, |_0, \text{accept})$,
$(\text{read}, |_1, \text{read}) \to 1(\text{read}, |_1, \text{read})(\text{read}, |_1, \text{read})$,
$(\text{read}, |_1, \text{accept}) \to 1(\text{read}, |_1, \text{read})(\text{read}, |_1, \text{accept})$,
$(\text{read}, |_1, \text{read}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, |_1, \text{read})$,
$(\text{read}, |_1, \text{accept}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, |_1, \text{accept})$.

Example (cont'd)

In the end, we have the following production rules. Let's trim them!

$(\text{read}, \$, \text{accept}) \to \varepsilon$,
$(\text{read}, |_1, \text{read}) \to 0$,
$(\text{read}, |_0, \text{read}) \to 1$,

$(\text{read}, \$, \text{accept}) \to 0(\text{read}, |_0, \text{read})(\text{read}, \$, \text{accept})$,

$(\text{read}, \$, \text{accept}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, \$, \text{accept})$,

$(\text{read}, \$, \text{accept}) \to 1(\text{read}, |_1, \text{read})(\text{read}, \$, \text{accept})$,

$(\text{read}, \$, \text{accept}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, \$, \text{accept})$.

$(\text{read}, |_0, \text{read}) \to 0(\text{read}, |_0, \text{read})(\text{read}, |_0, \text{read})$,
$(\text{read}, |_0, \text{accept}) \to 0(\text{read}, |_0, \text{read})(\text{read}, |_0, \text{accept})$,
$(\text{read}, |_0, \text{read}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, |_0, \text{read})$,
$(\text{read}, |_0, \text{accept}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, |_0, \text{accept})$,
$(\text{read}, |_1, \text{read}) \to 1(\text{read}, |_1, \text{read})(\text{read}, |_1, \text{read})$,
$(\text{read}, |_1, \text{accept}) \to 1(\text{read}, |_1, \text{read})(\text{read}, |_1, \text{accept})$,
$(\text{read}, |_1, \text{read}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, |_1, \text{read})$,
$(\text{read}, |_1, \text{accept}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, |_1, \text{accept})$.

Example (cont'd)

In the end, we have the following production rules. Let's trim them!

$(\text{read}, \$, \text{accept}) \to \varepsilon$,
$(\text{read}, |_1, \text{read}) \to 0$,
$(\text{read}, |_0, \text{read}) \to 1$,

$(\text{read}, \$, \text{accept}) \to 0(\text{read}, |_0, \text{read})(\text{read}, \$, \text{accept})$,

$(\text{read}, \$, \text{accept}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, \$, \text{accept})$,

$(\text{read}, \$, \text{accept}) \to 1(\text{read}, |_1, \text{read})(\text{read}, \$, \text{accept})$,

$(\text{read}, \$, \text{accept}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, \$, \text{accept})$.

$(\text{read}, |_0, \text{read}) \to 0(\text{read}, |_0, \text{read})(\text{read}, |_0, \text{read})$,
$(\text{read}, |_0, \text{accept}) \to 0(\text{read}, |_0, \text{read})(\text{read}, |_0, \text{accept})$,
$(\text{read}, |_0, \text{read}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, |_0, \text{read})$,
$(\text{read}, |_0, \text{accept}) \to 0(\text{read}, |_0, \text{accept})(\text{accept}, |_0, \text{accept})$,
$(\text{read}, |_1, \text{read}) \to 1(\text{read}, |_1, \text{read})(\text{read}, |_1, \text{read})$,
$(\text{read}, |_1, \text{accept}) \to 1(\text{read}, |_1, \text{read})(\text{read}, |_1, \text{accept})$,
$(\text{read}, |_1, \text{read}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, |_1, \text{read})$,
$(\text{read}, |_1, \text{accept}) \to 1(\text{read}, |_1, \text{accept})(\text{accept}, |_1, \text{accept})$.

Example (cont'd)

In the end, we have the following production rules. Let's trim them!

$(\text{read}, \$, \text{accept}) \to \varepsilon$,

$(\text{read}, |_1, \text{read}) \to 0$,

$(\text{read}, |_0, \text{read}) \to 1$,

$(\text{read}, \$, \text{accept}) \to 0(\text{read}, |_0, \text{read})(\text{read}, \$, \text{accept})$,

$(\text{read}, \$, \text{accept}) \to 1(\text{read}, |_1, \text{read})(\text{read}, \$, \text{accept})$,

$(\text{read}, |_0, \text{read}) \to 0(\text{read}, |_0, \text{read})(\text{read}, |_0, \text{read})$,

$(\text{read}, |_0, \text{accept}) \to 0(\text{read}, |_0, \text{read})(\text{read}, |_0, \text{accept})$,

$(\text{read}, |_1, \text{read}) \to 1(\text{read}, |_1, \text{read})(\text{read}, |_1, \text{read})$,

$(\text{read}, |_1, \text{accept}) \to 1(\text{read}, |_1, \text{read})(\text{read}, |_1, \text{accept})$,

Example (cont'd)

In the end, we have the following production rules. Let's trim them!

$(\text{read}, \$, \text{accept}) \to \varepsilon$,

$(\text{read}, |_1, \text{read}) \to 0$,

$(\text{read}, |_0, \text{read}) \to 1$,

$(\text{read}, |_0, \text{read}) \to 0(\text{read}, |_0, \text{read})(\text{read}, |_0, \text{read})$,

$(\text{read}, \$, \text{accept}) \to 0(\text{read}, |_0, \text{read})(\text{read}, \$, \text{accept})$,

$(\text{read}, |_1, \text{read}) \to 1(\text{read}, |_1, \text{read})(\text{read}, |_1, \text{read})$,

$(\text{read}, \$, \text{accept}) \to 1(\text{read}, |_1, \text{read})(\text{read}, \$, \text{accept})$,

Example (cont'd)

In the end, we have the following production rules. Let's trim them!

$(\text{read}, \$, \text{accept}) \to \varepsilon,$

$(\text{read}, |_1, \text{read}) \to 0,$

$(\text{read}, |_0, \text{read}) \to 1,$

$(\text{read}, |_0, \text{read}) \to 0(\text{read}, |_0, \text{read})(\text{read}, |_0, \text{read}),$

$(\text{read}, \$, \text{accept}) \to 0(\text{read}, |_0, \text{read})(\text{read}, \$, \text{accept}),$

$(\text{read}, |_1, \text{read}) \to 1(\text{read}, |_1, \text{read})(\text{read}, |_1, \text{read}),$

$(\text{read}, \$, \text{accept}) \to 1(\text{read}, |_1, \text{read})(\text{read}, \$, \text{accept}),$

Rewrite with simpler variable names if you want!

$$S \to 0OS \mid 1ZS \mid \varepsilon, \quad O \to 0OO \mid 1, \quad Z \to 1ZZ \mid 0.$$

Minimizing the grammar as we go

Could we have avoided generating the useless rules in the first place?

Minimizing the grammar as we go

Could we have avoided generating the useless rules in the first place?

Recall the intended meaning of the variable (p, A, q):

Minimizing the grammar as we go

Could we have avoided generating the useless rules in the first place?

Recall the intended meaning of the variable (p, A, q): words generated from it are those pronounced in the PDA when going from state p with A on top of the stack to state q with that A popped off the stack.

Minimizing the grammar as we go

Could we have avoided generating the useless rules in the first place?

Recall the intended meaning of the variable (p, A, q): words generated from it are those pronounced in the PDA when going from state p with A on top of the stack to state q with that A popped off the stack.

Therefore, if there is no way for the PDA to go from p to q while popping A, then (p, A, q) is useless and can be removed from the grammar (along with all rules using it).

Minimizing the grammar as we go

Could we have avoided generating the useless rules in the first place?

Recall the intended meaning of the variable (p, A, q): words generated from it are those pronounced in the PDA when going from state p with A on top of the stack to state q with that A popped off the stack.

Therefore, if there is no way for the PDA to go from p to q while popping A, then (p, A, q) is useless and can be removed from the grammar (along with all rules using it).

Let's look at the previous example again:

Minimizing the grammar as we go

Could we have avoided generating the useless rules in the first place?

Recall the intended meaning of the variable (p, A, q): words generated from it are those pronounced in the PDA when going from state p with A on top of the stack to state q with that A popped off the stack.

Therefore, if there is no way for the PDA to go from p to q while popping A, then (p, A, q) is useless and can be removed from the grammar (along with all rules using it).

Let's look at the previous example again: we could have predicted that $(\text{accept}, \$, \text{read})$ is useless for example.

Minimizing the grammar as we go

Could we have avoided generating the useless rules in the first place?

Recall the intended meaning of the variable (p, A, q): words generated from it are those pronounced in the PDA when going from state p with A on top of the stack to state q with that A popped off the stack.

Therefore, if there is no way for the PDA to go from p to q while popping A, then (p, A, q) is useless and can be removed from the grammar (along with all rules using it).

Let's look at the previous example again: we could have predicted that $(\text{accept}, \$, \text{read})$ is useless for example. As is $(\text{accept}, |_0, \text{read})$, $(\text{accept}, |_1, \text{read})$, $(\text{accept}, \$, \text{accept})$, etc.

Minimizing the grammar as we go

Could we have avoided generating the useless rules in the first place?

Recall the intended meaning of the variable (p, A, q): words generated from it are those pronounced in the PDA when going from state p with A on top of the stack to state q with that A popped off the stack.

Therefore, if there is no way for the PDA to go from p to q while popping A, then (p, A, q) is useless and can be removed from the grammar (along with all rules using it).

Let's look at the previous example again: we could have predicted that $(\text{accept}, \$, \text{read})$ is useless for example. As is $(\text{accept}, |_0, \text{read})$, $(\text{accept}, |_1, \text{read})$, $(\text{accept}, \$, \text{accept})$, etc.

Hence, while generating the rules, for each rule, ask yourself if each variable involved can actually be used in the PDA! If not, skip adding that rule.